

how_to_create_new_components

how to create new components

- "LAMW: Lazarus Android Module Wizard"
 - Author: Jose Marques Pessoa : jmpessoa__hotmail_com
 - [Repositório lazandroidmodulewizard](#)
 - [Forum lamw](#)

:: How to Create LAMW JNI Bridge Component code::

1. LAMW Component Model:

- :: jControl: No GUI Control
 - hint: you can use composition or inheritance model here....
- :: jVisualControl: GUI Control
 - warning: you MUST use inheritance model here.....

2. Creating a new component:

- 2.1. Right click Java tab:
 - 2.1. Insert a new [wrapper] Java or Kotlin class jControl/jVisualControl from template;
 - hint1. Alternatively, you can paste you java or kotlin code [template compatible] from clipboard!
 - hint2. Alternatively, you can import a "JAR" file and selec a class [at the moment, only for no visual control]!
 - hint3: You can also use the demo AppTryCode3 to produce a complete [wrapper] java class (outdated.. sorry) !
 - hint4. you do not need to build the "perfect" component just now later you can add more methods, events and properties! [sections 3 e 4 bellow]
 - warning/important: All "imports" from "Controls.java" template and new "imports" from your java/kotlin class code will be researched to try write the correct JNI methods signatures.....
- 2.2. Rename [and extends] the new class [and constructor], write some custom field, methods, etc...
 - [for "JAR" imported class this task is automated!]
 - guideline: please, preferentially init your news params names with "_", ex: int_flag, String_hello ...
 - java hint1: Only public methods will be translated to pascal [bridge] code.

- java hint2: use the mask `./` to make a public method invisible for the parse [ex. `./public myfunctio() {...}`]
- 2.3. In the java tab, If have only one java or kotlin class just right click it...
 - hint1: if you have some java method signature like this:
 - `public String GetCurrentHotspotSSID() throws IllegalAccessException {.....}`
 - Use only this part in code generation process [warning: save your original java code...]:
 - `public String GetCurrentHotspotSSID() {.....}`
 - note: the same is valid for Kotlin "fun"
 - hint2. save your java or kotlin class source code in LAMW folder "`.....\android_wizard\smartdesigner\java`" :
 - warning:
 - the LAMW java/kotlin to pascal converter dont support complex/generic types like "`java.util.Map<,>`", "`java.util.Set<>`" etc..
 - So, you need re-write the method signature [by hand] for primitives data/array type....
 - But simple java/kotlin object param [or return] is ok [including array!!!]
 - example:
 - `public void SetImage(Bitmap _image) { //<---- OK!`
 - `this.setImage(_image);}`
 - `public Bitmap GetImage(Bitmap _image) { //<---- OK! return this.getImage(); }`
 - warning/important: All "imports" from "Controls.java" template and new
 - "imports" from your java/kotlin class code will be researched to try write the correct JNI methods signatures.....
- 2.4. Popup Menu: Select "Write [Draft] Pascal jControl/jVisualControl Interface".
- 2.5. Right click "Pascal" tab and select "Register Component..." from Popup Menu;
 - warning: Mac users: please, you must copy the console app "lazres" [lazarus/tools] to the folder "`.../LazAndroidWizard/ide_tools`";
- 2.6. [Dialog]: select a 16x16 PNG icon file;
- 2.7. [Dialog]: open a "register_*.pas" file (ex. "register_extra.pas"); **
 - hint: you can select "register_template.pas" to write a custom register file, but, then, you must insert it into "tfpandroidbridge_pack.lpk".
- 2.8. Ok. You got your DRAFT JNI pascal component stuff! Please, look in the folder "`.../LazAndroidWizard/android_bridges`" directory:
 - `:: *.pas <---` open and fix/complete the generated pascal code with your others events/properties/methods/variables/overload, etc..!
 - `:: *_icon.lrs <---` new icon resource

- :: register_extras.pas <--- by default, the new component will be included here...
- 2.9. [Lazarus IDE Menu]: Package -> Open Package File (*.lpk) --> "tfpandroidbridge_pack.pas" in folder "...../LazAndroidWizard/android_bridges".
- 2.10. [Package Wizard]: Compile ---> Use ---> Install.
 - Warning: to compile/install/reinstall a LAMW package in Lazarus/Laz4Android [windows], please, open a "dummy" pure windows project.... you MUST always close the cross compile project!
- 2.11. Wait Lazarus re-building ...
- 2.12. The new component is on "Android Bridges Extra" or other Lazarus component palette...
- 3. Added more methods to your component
 - 3.1. write new method to your ".java" our ".kt" code

```
ex.  
  
public void SetFoo(int _foo) {  
    mFoo = foo;  
}  
public int GetFoo() {  
    return mFoo;  
}  
  
or:  
  
fun SetFoo(_foo: Int) {  
    mFoo = _foo;  
}  
  
fun GetFoo(): Int {  
    return mFoo;  
}
```

- 3.1. In java tab write a draft java or kotlin class code to wrapper the [new] methods;
 - ex

```
class jMyComponent {

    public void SetFoo(int _foo) {
        mFoo = _foo;
    }

    public int GetFoo() {
        return mFoo;
    }

}
```

- or kotlin: append a pair "()" after the class name!

```
class jMyComponent() {

    fun SetFoo(_foo: Int) {
        mFoo = _foo;
    }

    fun GetFoo(): Int {
        return mFoo;
    }

}
```

- 3.2 Right click the Java tab and select "Write [draft] complementary Pascal Interface"
- 3.4 Copy the the pascal interface code to your component unit!!!
- 2. Added "native" [events] methods to your component
 - 4.1. write the native method "call" to to your ".java" or ".kt" code [study some LAMW code as example!]
 - ex1: [from jSpinner.java]

```
controls.pOnSpinnerItemSelected(pascalObj, position, caption);
//return void
```

- ex2: [fiction...]

```
int i =
controls.pOnSpinnerItemSelected(pascalObj, position, caption);
//Return int
```

- ex3: [fiction...]

```
int[] v;
v =
controls.pOnSpinnerItemSelected(pascalObj, position, caption);
//return int[]
for (int i = 0; i < v.length; i++) {
    //Log.i("v["+i+"]", ""+v[i]);    //do some use...
}
```

- ex4: [fiction...]

```
String[] s;
s =
controls.pOnSpinnerItemSelected(pascalObj, position, caption);
//return String[]
for (int i = 0; i < s.length; i++) {
    //Log.i("s["+i+"]", ""+s[i]);    //do some use...
}
```

note: "controls" is the LAMW object/class that handle native methods for your component.....

- 4.2. write the native method signature to your [component] in "*.native" file [study some LAMW code as example!]

- ex1. [from jSpinner.native]

```
public native void pOnSpinnerItemSelected(long pasobj,
int position, String caption);
```

- ex2. [fiction...]

```
public native int pOnSpinnerItemSelected(long pasobj,
int position, String caption);
```

- ex3. [fiction...]

```
public native int[] pOnSpinnerItemSelected(long pasobj,
int position, String caption);
```

- ex4. [fiction...]

```
public native String[] pOnSpinnerItemSelected(long
pasobj, int position, String caption);
```

- 4.3. In "java tab" form write a draft/dummy class code to wrapper the natives methods using "java" sintaxe..... even if you are coding a kotlin class !

```
class jMyComponent {

    public native void pOnMyComponentItemSelected(long
pasobj, int position, String caption); //MyComponent
    public native void pOnMyComponentItemRemoved(long
pasobj, int position); //MyComponent

}
```

- NOTE1: the "//MyComponent" signs the part of the method name that will be overshadowed in the nomination of the property.

```
// ex.
FOnItemSelected [ not FOnMyComponentItemSelected ]
```

- 4.4 Right click the Java tab and select "Write [draft] complementary Native Method Pascal Interface"
- 4.5 Copy the the pascal interface code [generated] to your pascal component unit [mycomponent.pas] and to "Laz_And_Controls_Events.pas"!!!
- Congratulations!!!
 - Thank you!
- //simple java code example for simple test...:

```
/* jControl template example: */
class jMyHello /*extends ...*/ {

    private long    pascalObj = 0;      // Pascal Object
    private Controls controls = null;   // Control Class for
events
    private Context context    = null;

    private int mFlag;                // <----- my custom field
    private String mHello;            // <----- my custom field
    private int[] mBufArray;          // <----- my custom field

    public jMyHello(Controls contrls, long Self, int _flag,
    String mHello) { //Add more arguments if needed from event handler
```

```
String _hello) { //Add more params if needed [preferentially init  
your new param name with "_", ex: int _flag, String _hello]!  
  
    //super(contrls.activity);  
  
    context    = contrls.activity;  
    pascalObj  = Self;  
    controls   = contrls;  
  
    mFlag = _flag;  
    mHello = _hello;  
    mBufArray = nul;  
  
}  
  
public void jFree() {  
    //free local objects...  
    mBufArray = nul;  
}  
  
    //write others [public] methods code here..... <----- customs  
methods  
  
    //Warning: please, preferentially init your news params names  
with "_", ex: int _flag, String _hello ...  
  
    public void SetFlag(int _flag) { // <----- my custom method  
        mFlag = _flag;  
    }  
  
    public int GetFlag { // <----- my custom method  
        return mFlag;  
    }  
  
    public void SetHello(String _hello) { // <----- my custom  
method  
        mHello = _hello;  
    }  
  
    public String getHello {// <----- my custom method  
        return mHello;  
    }  
  
    public int[] GetSumIntArray(int[] _v1, int[] _v2, int _size)  
    {// <----- my custom method  
  
        mBufArray[] = new int[_size];  
  
        for (int i=0, i < size, i++) {  
            mBufArray[i] = _v1[i] + _v2[i];  
        }  
  
        return mBufArray;  
    }  
}
```

```
public void ShowHello() { // <<----- my custom method  
  
    Toast.makeText(controls.activity, mHello,  
    Toast.LENGTH_SHORT).show();  
  
}  
  
}
```

